Aula 52 — Serverless: Funções, Eventos e Provedores

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Objetivos da Aula

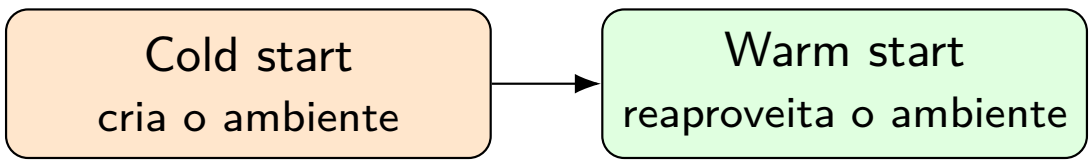
- Compreender a **arquitetura orientada a eventos**.
- Conhecer o **ciclo de execução** de uma função.
- Comparar serviços **serverless** de provedores.
- Aplicar boas práticas de serverless.

Arquitetura Orientada a Eventos

- Componentes **reagem** a eventos, em vez de esperar requisições.
- **Desacoplamento** entre produtores e consumidores.
- **Filas e streams** (SQS, Kinesis, Pub/Sub).
- Escala e resiliência naturais.

Ciclo de Execução

- **Cold start:** primeira invocação cria o ambiente.
- **Warm start:** invocações seguintes reaproveitam o ambiente.
- **Timeout:** limite de tempo de execução.
- **Memória** define também a CPU alocada.



Padrões Comuns

- **API + funções** (serverless backend).
- **Event-driven ETL**.
- **Fan-out** (um evento, várias funções).
- **Orquestração** de funções (Step Functions / Durable Functions).

Provedores e Serviços

Provedor	FaaS	API	Fila/Stream
AWS	Lambda	API Gateway	SQS/Kinesis
Azure	Functions	API Management	Service Bus
Google	Cloud Functions	API Gateway	Pub/Sub

Boas Práticas

- Funções **pequenas e específicas**.
- **Idempotência** no processamento de eventos.
- Gerenciar **segredos** fora do código.
- **Observabilidade** (logs, métricas, tracing).
- Controlar **concorrência**.

Segurança

- **IAM** com menor privilégio por função.
- Isolamento de **rede** e acesso a recursos.
- Proteção de **variáveis de ambiente** sensíveis.
- Monitoramento de execuções anômalas.

Custos

- Cobrança por **invocações** e **tempo** (ms).
- Filas e API podem ter custo separado.
- Monitorar para evitar **laços** infinitos.
- Comparar com contêiner para cargas constantes.

Tip

Para cargas **constantes**, contêineres podem sair mais baratos; para cargas **intermitentes**, serverless tende a vencer.

Resumo da Aula

- Serverless favorece arquiteturas **orientadas a eventos**.
- Conhecer **cold/warm start** e limites é essencial.
- Cada provedor tem serviços equivalentes.
- Boas práticas: funções pequenas, idempotência e observabilidade.

Atividade Rápida

- Explique **cold start** e **warm start**.
- Descreva um padrão **event-driven** de ETL.
- Compare o custo de serverless x contêiner.

Referências

- AWS Lambda / Azure Functions / Cloud Functions.

Próxima Aula

- **Aula 53:** Atividade — Computação sem Servidor.